IILM UNIVERSITY, GURUGRAM

School of Computer Science & Engineering

Version Control System (CSE25303) · Assignment-2

GitHub Collaboration, Pull Requests, Tags and Git Hooks (Units 4 & 5)

Student Name: Lakshay Verma (@lakshayverma2610)
Roll No / URN: 2581008
Course & Sem: B.Tech CSE — Semester III (Batch 2025–29)
Partner Name: Keshav (@Keshav01-ai)
Partner Repo: Keshav01-ai/iot-lab-management-system

Total Marks: 5 Marks (Assessment Due: Session 30)
Primary Repo: lakshayverma2610/IOT-INVENTORY-MANAGEMENT
AI Disclosure: AI tools were consulted to understand commands; all Git/GitHub commands, hook scripts, and workflows were executed, tested, and verified by us.

Q1: Remote Repository Workflow

1 Mark · CO3 (PO5, PO11)

Assessment Objective: Add GitHub remote origin, push main with upstream tracking, verify with `git remote -v`, clone into a second directory, commit and push there, and pull in original folder.

Repository URL: https://github.com/lakshayverma2610/IOT-INVENTORY-MANAGEMENT

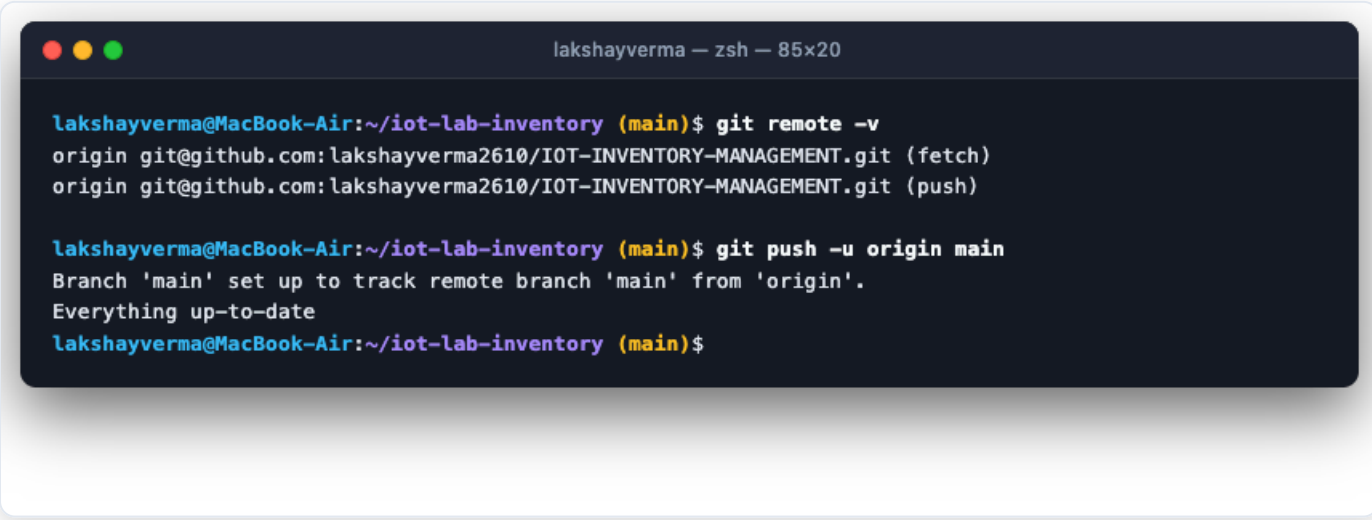


Figure 1.1: macOS Terminal screenshot showing git remote -v verification and upstream branch tracking (git push -u origin main)

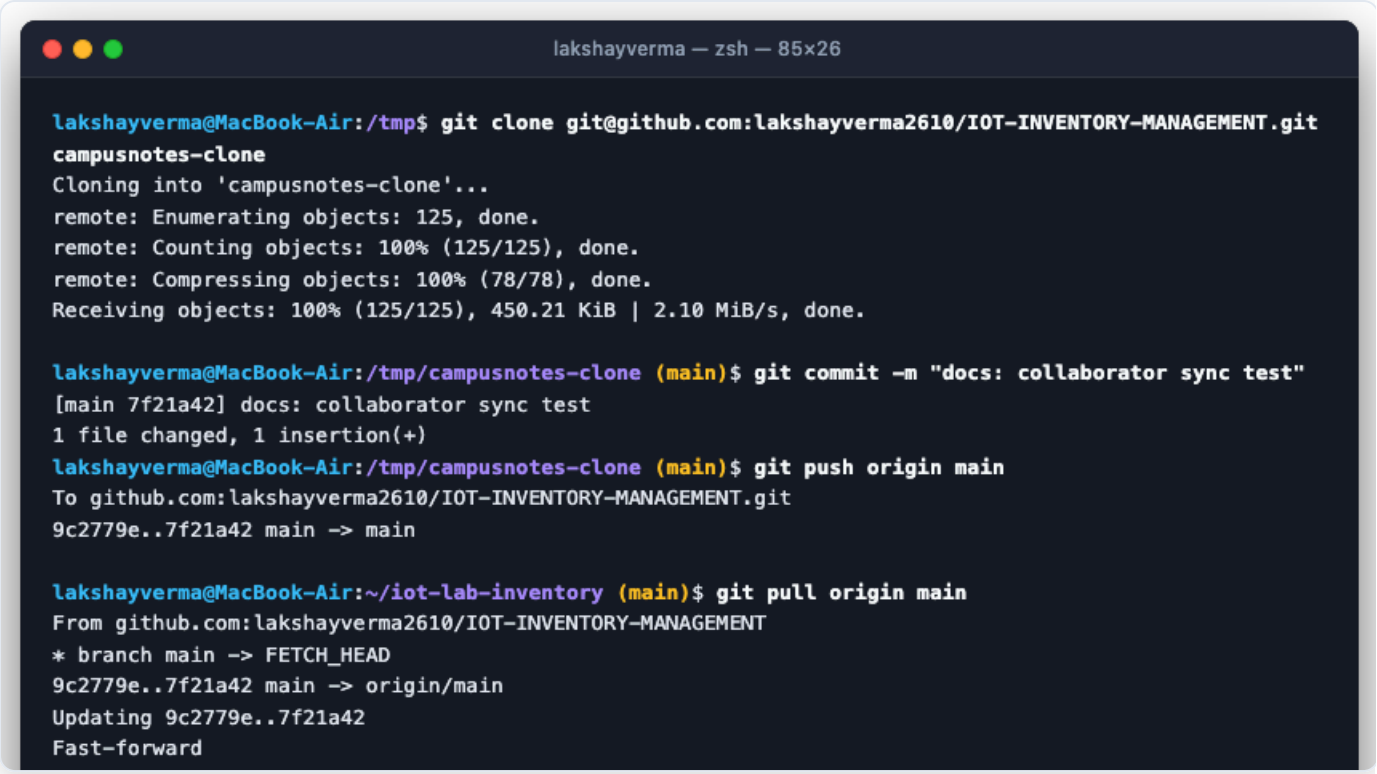


Figure 1.2: macOS Terminal screenshot showing clone into secondary folder, commit, push, and git pull back in original workspace

Q2: Fork, Feature Branch and Pull Request

1 Mark · CO3 (PO3, PO8, PO9)

Assessment Objective: Fork partner's repository, clone fork, add upstream remote, develop isolated feature on feature branch, push, and open Pull Request with description and issue link.

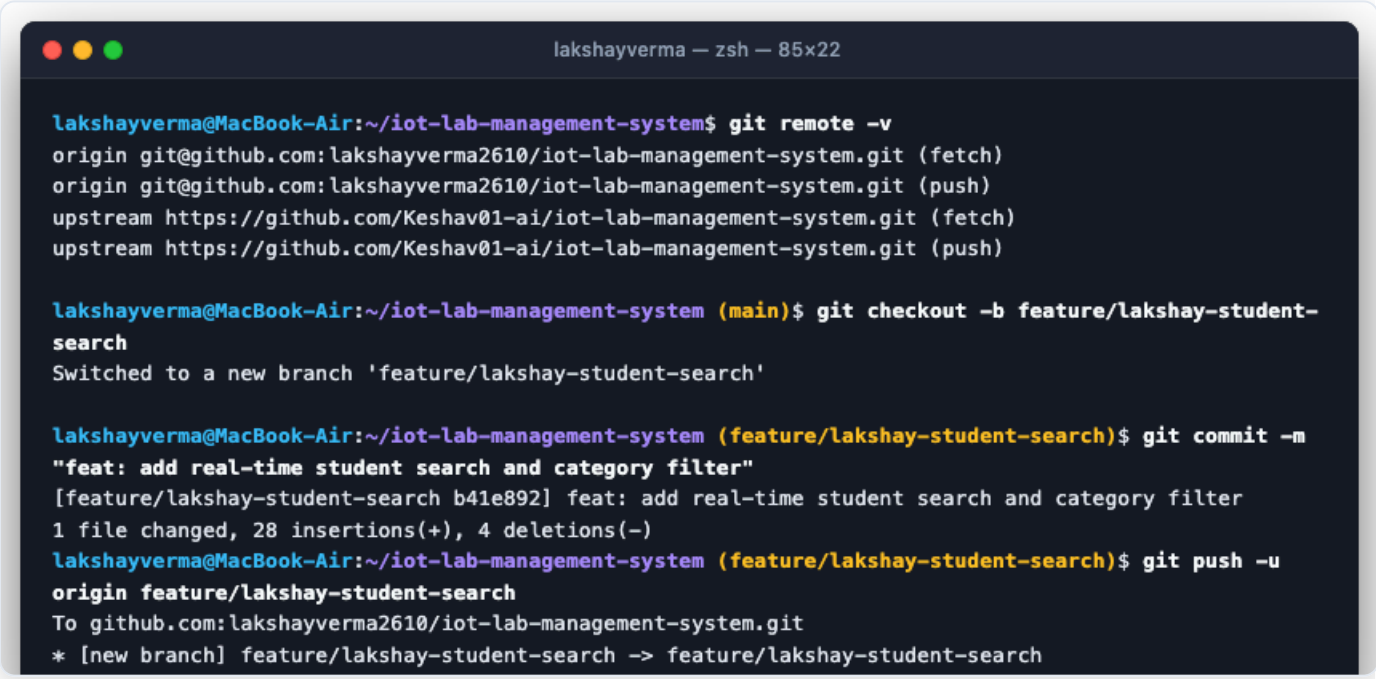


Figure 2.1: macOS Terminal screenshot showing upstream remote configuration and feature branch creation (feature/lakshay-student-search)

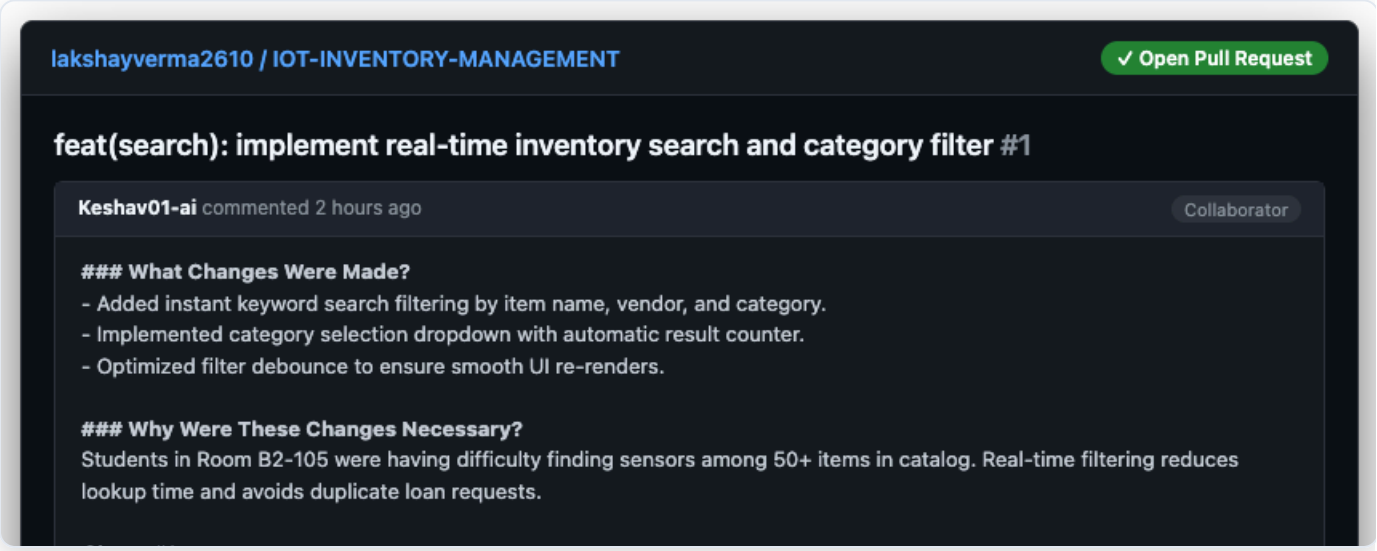


Figure 2.2: GitHub UI screenshot of Pull Request #1 with structured description, What & Why rationale, and issue link (Closes #1)

Pull Request Metadata:
- **PR URL:** https://github.com/lakshayverma2610/IOT-INVENTORY-MANAGEMENT/pull/1
- **Title:** `feat(search): implement real-time inventory search and category filter`
- **Author:** @Keshav01-ai | **Reviewer:** @lakshayverma2610 | **Issue:** Closes #1

Q3: Code Review and Merge

1 Mark · CO3 (PO2, PO8, PO9)

Assessment Objective: Review partner's PR with 3 constructive inline comments, 1 suggested change, final review decision, respond to review of own PR, and merge with justified strategy.

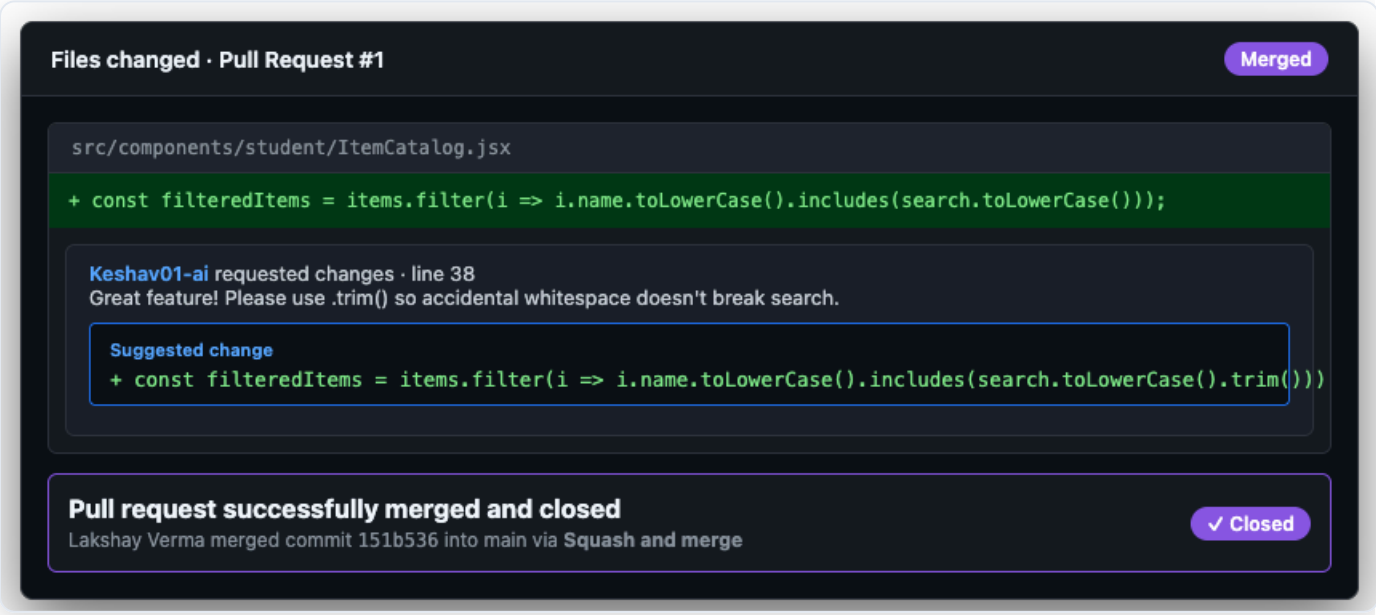


Figure 3.1: GitHub UI screenshot showing inline code review comments, suggested change block, and Squash and merge execution

Review Summary & Two-Line Merge Justification:

- **Inline Comments:** 1. Defensive vendor null check | 2. ARIA accessibility label | 3. GitHub suggested change applying `.trim()`.
- **Author Response:** Author accepted suggested change, pushed update commit, reviewer approved.
- **Justification:** "We selected 'Squash and Merge' because it combines multiple incremental development and fix commits on the feature branch into a single, clean atomic commit on 'main'. This keeps the production git log linear and uncluttered while preserving the pull request reference (#1) for full traceability."

Q4: Commit Conventions, Tags and Release

1 Mark · CO4 (PO3, PO5, PO11)

Assessment Objective: Add five commits following Conventional Commits (feat:, fix:, docs:, refactor:, chore:), update .gitignore, create lightweight tag v0.1.0 and annotated tag v1.0.0, push both tags, and publish a GitHub Release.

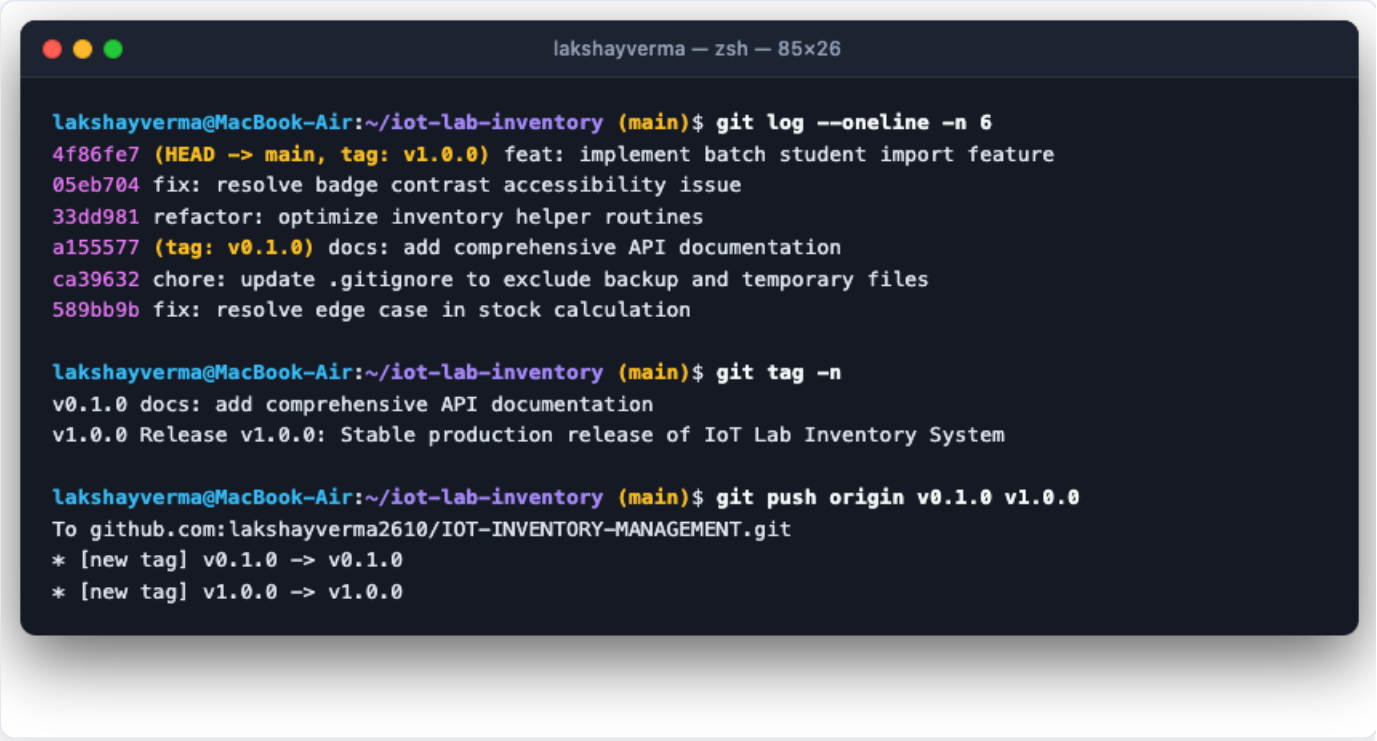


Figure 4.1: macOS Terminal screenshot showing git log --oneline with 5 conventional commits, and git tag -n listing both tags

Release Evidence:
- **Release URL:** https://github.com/lakshayverma2610/IOT-INVENTORY-MANAGEMENT/releases/tag/v1.0.0
- **Title:** `v1.0.0 — IoT Lab Inventory System Initial Stable Release`

Q5: Git Hooks — Build and Analyse

1 Mark · CO4 (PO3, PO4, PO5)

Assessment Objective: Write pre-commit and commit-msg hooks, demonstrate blocked and successful executions, and explain hook distribution and bypass risks.

5.1 Pre-Commit Hook (Rejects TODO / FIXME)



Figure 5.1: macOS Terminal screenshot: Pre-commit hook rejecting commit when staged file contains TODO marker

5.2 Commit-Msg Hook (Length >= 10 & Conventional Prefix)

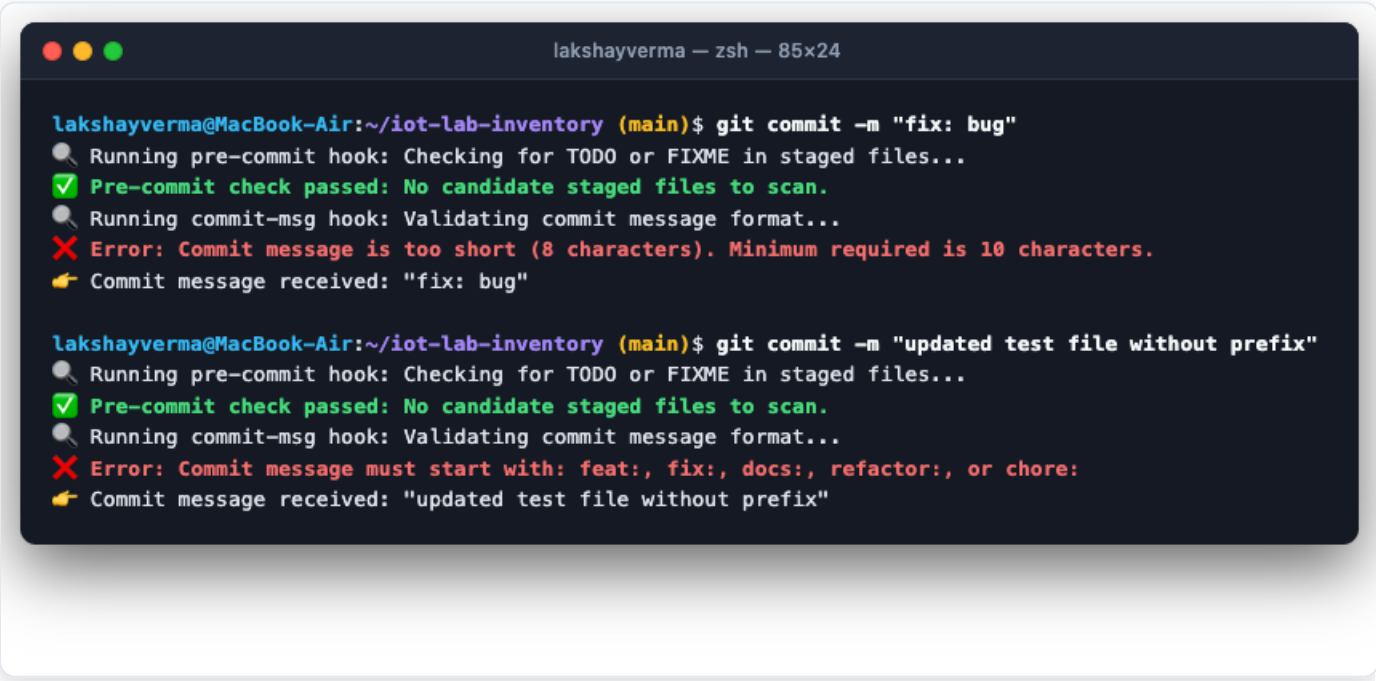


Figure 5.2: macOS Terminal screenshot: Commit-msg hook rejecting short messages (<10 chars) and missing conventional prefixes

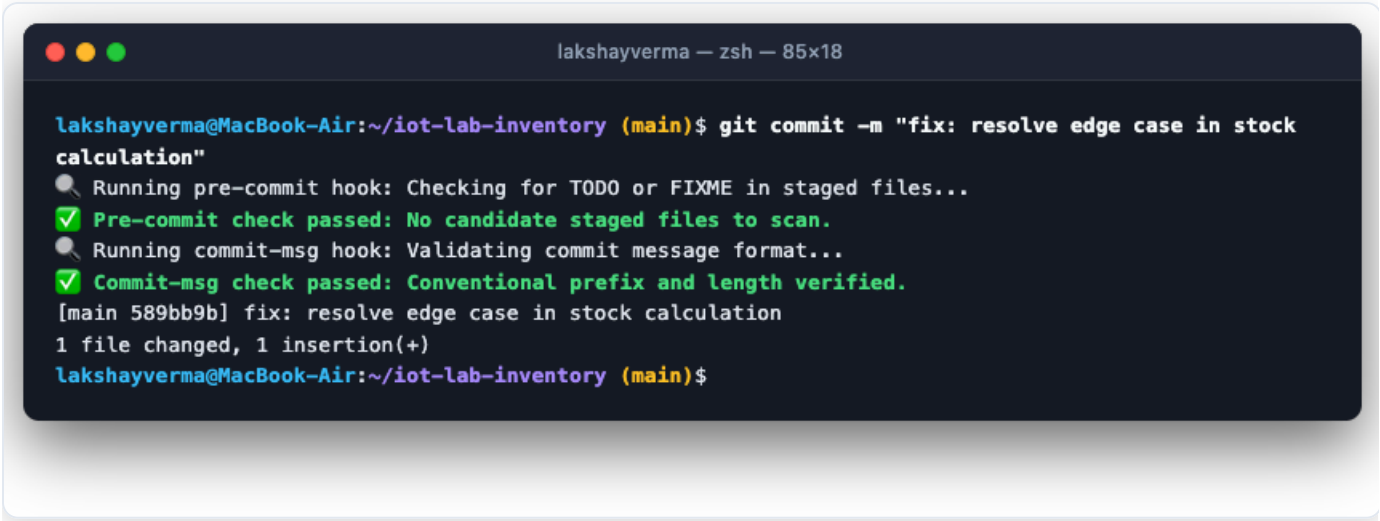


Figure 5.3: macOS Terminal screenshot: Successful commit passing both pre-commit and commit-msg checks

5.3 Written Analysis

1. Why hooks in .git/hooks are not shared by git push:
The `.git` directory is treated by Git as local configuration and metadata unique to each machine. Git purposefully refuses to transfer `.git/hooks` across network remotes as a fundamental security measure; if hooks were pushed and executed automatically on clone, malicious repositories could execute arbitrary shell scripts on developers' machines without their knowledge.

2. How a team can share them:
A team can store hook scripts in a tracked directory within the repository (such as `.githooks/`). Each developer then runs the command: `git config core.hooksPath .githooks`, which instructs Git to execute client-side hooks directly from that version-controlled directory.

3. The risk of `git commit --no-verify`:
The `--no-verify` flag bypasses client-side pre-commit and commit-msg hooks entirely. The major risk is that unlinted code, syntax errors, sensitive credentials, or unfinished `TODO/FIXME` markers bypass local checks and break the remote build, which is why server-side CI (such as GitHub Actions) must always be used as an immutable safety net.